实验6 人脸识别系统使用

姓名	年级专业	课程名称	实验时间
姚嫣如	23 网络空间安全	网络安全技术	2026.4.17

一、实验目的

1. 部署并运行基于机器学习的人脸识别系统，了解 OpenCV、Dlib 等工具在人脸检测、人脸特征提取和人脸识别中的基本应用，了解生物特征身份认证的基本原理。
2. 完成人脸信息采集、人脸特征提取和实时识别功能，实现摄像头采集的人脸与已录入人脸特征数据库之间的匹配与验证。
3. 在人脸识别系统基础上设计并实现简易登录界面，将口令认证或证书认证与人脸识别相结合，构建双因子认证系统，理解双因子认证的工作流程及其安全优势。

二、实验要求

1. **人脸识别系统部署**：在物理机环境中部署基于 Dlib 和 OpenCV 的人脸识别项目，完成依赖安装、模型配置及系统运行测试。
2. **人脸信息采集与特征提取**：使用摄像头采集用户人脸图像，生成人脸特征数据，并建立人脸特征数据库。
3. **实时人脸识别**：调用摄像头进行实时人脸检测与识别，实现已录入用户的人脸身份验证。
4. **双因子认证实现**：设计并实现简易登录界面，将口令认证或证书认证与人脸识别认证相结合。
5. **认证流程要求**：
    - 第一阶段完成用户名密码认证（或证书认证）；
    - 第二阶段完成人脸识别认证；
    - 两个认证环节均通过后方可登录成功。
6. **功能验证**：能够正确处理登录成功、密码错误、证书验证失败、人脸认证失败等情况，并展示完整认证流程。
7. **安全性分析**：分析双因子认证机制相较于单一口令认证在身份验证安全性方面的优势。

三、实验环境

软硬件环境：

- 操作系统：Windows 11
- 硬件设备：带摄像头的物理机
- Python 环境管理工具：Miniconda
- Conda 虚拟环境名称：face_lab
- Python 版本：Python 3.8
- 开发工具：Visual Studio Code / 命令行终端

依赖库：

- OpenCV：用于摄像头调用、图像读取、视频帧处理、人脸框绘制和窗口显示；
- Dlib：用于人脸检测、68 点人脸关键点定位以及 128 维人脸特征向量提取；
- NumPy：用于人脸特征向量的数值计算和欧氏距离计算；
- Pandas / CSV：用于读取和保存人脸特征数据库文件；
- Pillow：用于图像文件读取和格式转换；
- Tkinter：用于实现简易图形化登录界面；

四、实验步骤

（一）人脸识别系统

1. **基本流程**

   摄像头采集画面 → 人脸检测 → 人脸关键点定位 → 人脸特征提取 → 与已注册人脸库比对 → 输出识别结果。

   - 支持摄像头人脸录入、建立人脸数据库，以及实时识别单人或多人脸；
   - 当前项目还提供了 Tkinter 录入界面、OpenCV 录入界面、普通识别、单人优化识别和 OT 跟踪优化识别等多个脚本。

   人脸检测的三个模型/函数：

   - **人脸检测器**：项目中使用 `dlib.get_frontal_face_detector()`，这是基于 HOG 特征的正脸检测器，用来从摄像头画面中找出人脸矩形框。
   - **人脸关键点检测器**：项目使用 `shape_predictor_68_face_landmarks.dat`，它会在人脸区域中定位眼睛、鼻子、嘴巴、轮廓等 68 个关键点。
   - **人脸特征描述子模型**。项目使用 `dlib_face_recognition_resnet_model_v1.dat`，把

一张人脸映射成一个 **128 维特征向量**。之后程序会把摄像头中当前人脸的 128 维向量与数据库中已有人员的 128 维向量进行欧氏距离比较，距离越小，代表越像。

2. 人脸注册与识别数据库

数据库形式如下：

```
data/
├── data_faces_from_camera/      # 摄像头采集的人脸图片
│   ├── person_1/
│   ├── person_2/
│   └── ...
└── features_all.csv             # 每个人的人脸特征向量
```

- 运行人脸录入脚本后，系统会把拍到的人脸图片保存到 `data/data_faces_from_camera`；
- 然后运行特征提取脚本，把每个人的人脸图片转成 128 维特征，并保存到 `features_all.csv`，该文件规模为 `n * 129`，其中 `n` 是录入人数，129 列包括“人名 + 128 维特征”。

3. 双因子认证

```
第一因子：know sth
例如：用户名 + 密码

第二因子：be sth
例如：摄像头识别到的人脸
```

4. 实现逻辑

- 单独跑通 GitHub 人脸识别项目
- 把人脸识别逻辑封装成一个函数，例如 face_auth()
- 写一个简易登录界面，先校验密码，再调用 face_auth()

完整流程：

```
用户输入用户名和密码
        ↓
密码验证通过？
        ↓ 是
打开摄像头进行人脸识别
        ↓
识别到当前用户本人？
        ↓ 是
登录成功
```

（二）项目部署

1. **创建 Conda 环境**

   打开 **Anaconda Prompt** 或 **PowerShell**：

```
conda create -n face_lab python=3.8 -y
conda activate face_lab
```

   Dlib 在新版本 Python 上更容易出现依赖兼容问题，因此使用 Python3.8或3.9。

```
Downloading and Extracting Packages:

Preparing transaction: done
Verifying transaction: done
Executing transaction: done
#
# To activate this environment, use
#
#     $ conda activate face_lab
#
# To deactivate an active environment, use
#
#     $ conda deactivate


Channel "defaults" has the following notices:
  [info] -- Tue Jun  9 00:00:00 2026
  PyTorch 2.12 with CUDA support is now available to install with your current channel (Anaconda Mas
```

```
(base) C:\Users\RMX08>conda activate face_lab

(face_lab) C:\Users\RMX08>
```

2. **安装基础依赖**

   用 conda-forge 安装 `dlib`，比直接 `pip install dlib` 稳定。

```
conda install -c conda-forge dlib opencv numpy pandas pillow scikit-image
cmake -y

# 安装项目其他依赖
pip install imutils
```

```
## Package Plan ##

  environment location: D:\miniconda3\envs\face_lab

  added / updated specs:
    - cmake
    - dlib
    - numpy
    - opencv
    - pandas
    - pillow
    - scikit-image
```

```
(face_lab) D:\yyr\study\aaprofessional\network_security_technology\lab6>pip install imut
ils
Collecting imutils
  Downloading imutils-0.5.4.tar.gz (17 kB)
  Preparing metadata (setup.py) ... done
Building wheels for collected packages: imutils
  Building wheel for imutils (setup.py) ... done
  Created wheel for imutils: filename=imutils-0.5.4-py3-none-any.whl size=25857 sha256=9
3cca121bc0f2aa90532c0dd17d2a5a95f3dbfb2b25f9ae0134bf07246dba5f7
  Stored in directory: c:\users\rmx08\appdata\local\pip\cache\wheels\59\1b\52\0dea905f82
78d5514dc4d0be5e251967f8681670cadd3dca89
Successfully built imutils
Installing collected packages: imutils
Successfully installed imutils-0.5.4
```

测试conda python自带的Tkinter是否可以使用

```
python -c "import tkinter; print('tkinter ok')"
```

```
(face_lab) D:\yyr\study\aaprofessional\network_security_technology\lab6>python -c "impor
t tkinter; print('tkinter ok')"
tkinter ok
```

3. 下载项目

进入实验文件夹下克隆项目

```
git clone https://github.com/coneypo/Dlib_face_recognition_from_camera.git

cd Dlib_face_recognition_from_camera
```

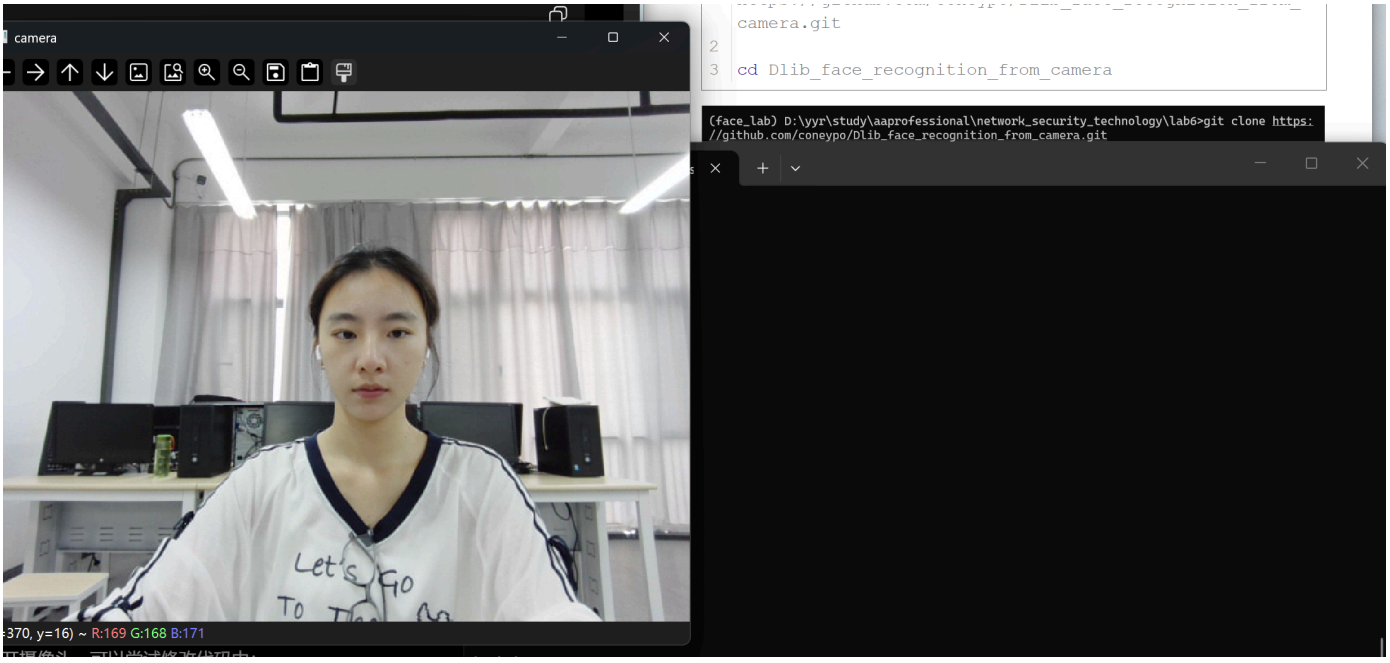
```
(face_lab) D:\yyr\study\aaprofessional\network_security_technology\lab6>git clone https://github.com/coneypo/Dlib_face_recognition_from_camera.git
Cloning into 'Dlib_face_recognition_from_camera'...
remote: Enumerating objects: 1089, done.
remote: Counting objects: 100% (128/128), done.
remote: Compressing objects: 100% (32/32), done.
remote: Total 1089 (delta 113), reused 96 (delta 96), pack-reused 961 (from 3)
Receiving objects: 100% (1089/1089), 128.20 MiB | 1.09 MiB/s, done.
Resolving deltas: 100% (441/441), done.
Updating files: 100% (25/25), done.
```

4. **测试摄像头是否能被OpenCV调用**

运行项目自带的摄像头测试脚本：

```
python how_to_use_camera.py
```

弹出摄像头窗口，说明环境基本正常：



5. **录入人脸**

项目当前 README 提供两种录入方式：

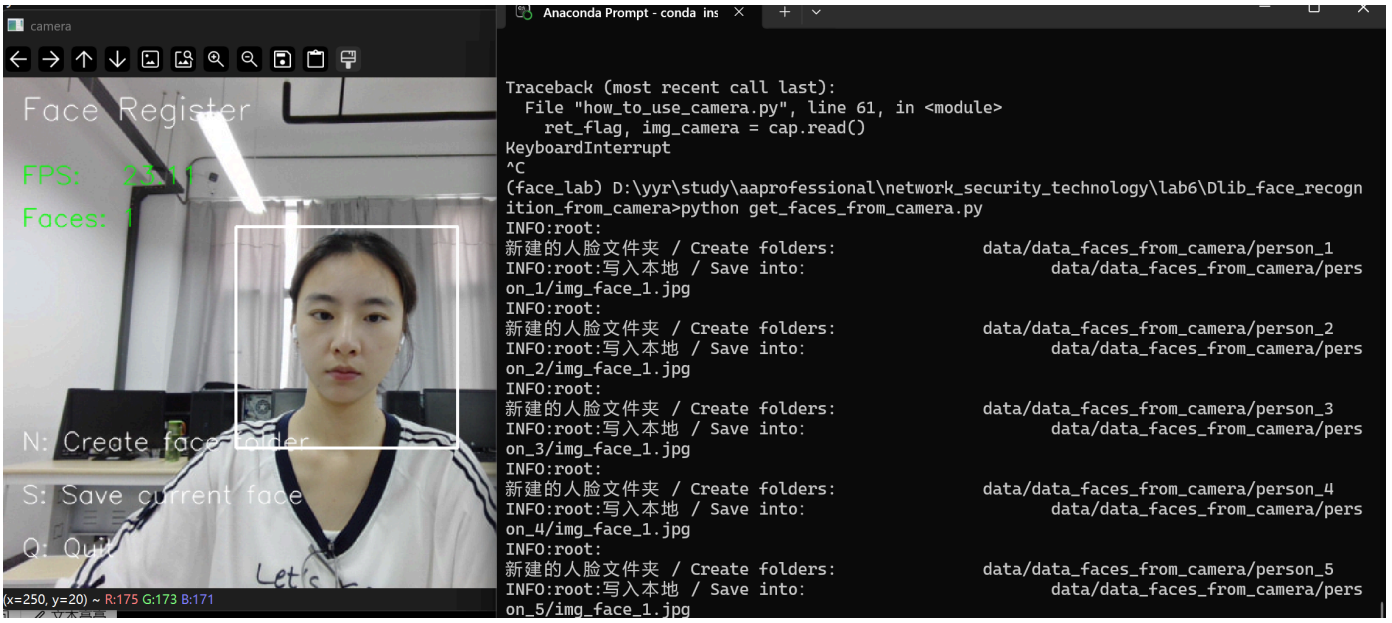
- Tkinter GUI，Tkinter 版本支持录入时设置中文姓名。
- OpenCV GUI；

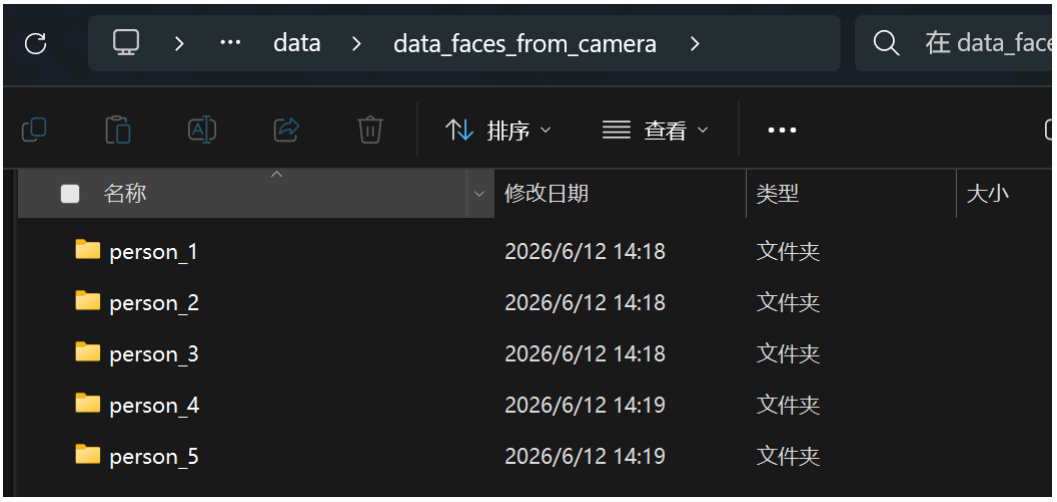
使用OpenCV 版本，简单稳定：

```
python get_faces_from_camera.py
```

录入时要先按 N 新建人员文件夹，再按 S 保存人脸图片；并且人脸不要离摄像头太近，否则可能出现“OUT OF RANGE”。

每个人保存 5 到 10 张图片，角度略有变化，但不要太夸张。





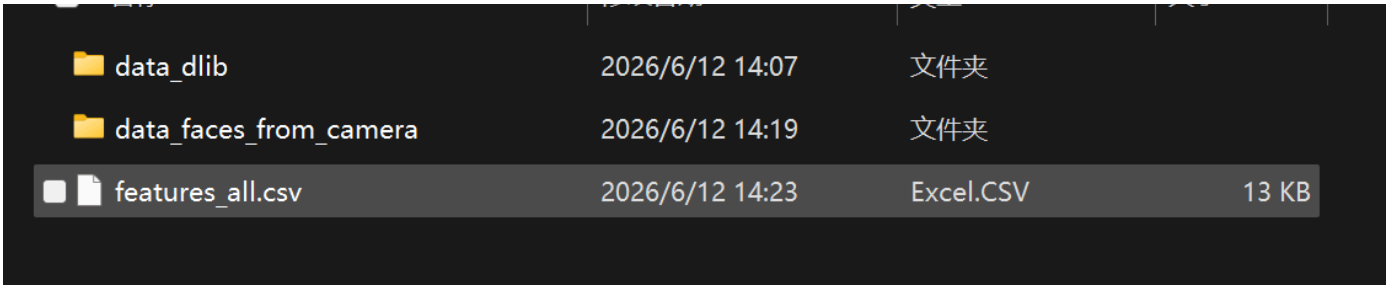
6. 提取人脸特征

录入完成后运行特征提取脚本：

```
python features_extraction_to_csv.py
```

```
(face_lab) D:\yyr\study\aaprofessional\network_security_technology\lab6\Dlib_face_recognition_from_camera>python features_extraction_to_csv.py
INFO:root:data/data_faces_from_camera/person_person_1_yolanda
INFO:root:正在读的人脸图像 / Reading image: data/data_faces_from_camera/person_1_yolanda/img_face_1.jpg
INFO:root:检测到人脸的图像 / Image with faces detected: data/data_faces_from_camera/person_1_yolanda/img_face_1.jpg
INFO:root:
```

成功后在 data 文件夹下生成文件 `features_all.csv`



这个文件就是后续识别使用的人脸特征库。

7. 调用摄像头进行实时识别

可以使用如下脚本：

```
# 普通识别版本，对每一帧都做检测和识别：
python face_reco_from_camera.py

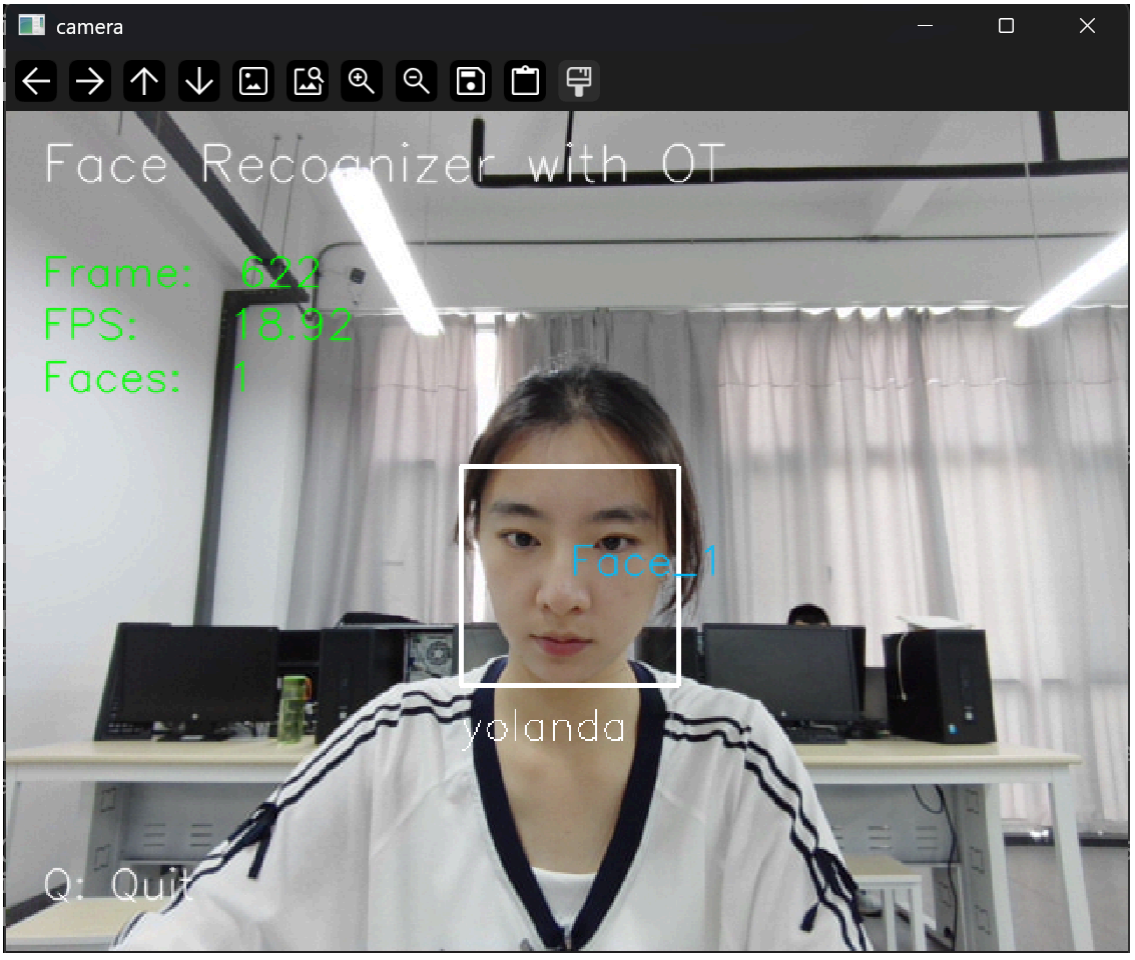
# 优化版本，适合人脸数小于等于 1 的场景
python face_reco_from_camera_single_face.py

# 优化版本2，使用 OT 跟踪来提高 FPS。
python face_reco_from_camera_ot.py
```

> **Note**
>
> FPS 全称是 **Frames Per Second**，每秒帧数，表示程序每秒能处理多少张画面。摄像头视频本质上是一张张连续图片：
>
> - 1 秒处理 5 张画面 → 5 FPS，比较卡
> - 1 秒处理 15 张画面 → 15 FPS，基本可用
> - 1 秒处理 30 张画面 → 30 FPS，比较流畅
>
> 在人脸识别程序里，每一帧大概要做这些事情：
>
> - 读取摄像头画面
> - 检测画面中有没有人脸
> - 提取人脸关键点
> - 提取 128 维人脸特征
> - 和数据库里的人脸特征比较
> - 在画面上显示识别结果

> **Note**
>
> OT 通常指 **Object Tracking**，目标跟踪，作用是：**不必每一帧都重新做人脸检测和人脸识别**，而是在识别到人脸之后，后续几帧直接“跟踪”这个人脸的位置。
>
> 普通版本：

> - 第 1 帧：检测人脸 + 识别人脸
> - 第 2 帧：检测人脸 + 识别人脸
> - 第 3 帧：检测人脸 + 识别人脸
>
> OT 跟踪版本：
>
> - 第 1 帧：检测人脸 + 识别人脸
> - 第 2 帧：跟踪上一帧的人脸位置
> - 第 3 帧：跟踪上一帧的人脸位置
>
> 隔一段时间再重新检测和识别，“跟踪”通常比“重新检测 + 重新识别”快很多，减少重复计算，所以可以提高 FPS。



（三）登陆界面

1. **登陆系统流程**
   - 输入用户名和密码
   - 密码正确后打开摄像头
   - 识别到当前用户本人
   - 登录成功

- 否则登录失败

实现过程即：

- 把“人脸识别程序”封装成一个**函数**
- 让它返回 **True 或 False**
- 然后和密码组合起来

2. 代码逻辑

（1）密码校验

由于本次实验着重体验双因子认证，因此使用简化的简单字典来明文保存用户名和密码，暂不考虑保存密码哈希。

```
USERS = {
    "yolanda": {
        "password": "123456",
        "face_name": "yolanda"
    }
}
```

（2）人脸识别

把识别逻辑封装成函数：

```
def face_auth(expected_name, timeout_seconds=10):
    """
    读取 features_all.csv
    打开摄像头
    检测当前画面中的人脸
    提取 128 维人脸特征
    和数据库中的人脸特征计算欧氏距离
    距离小于阈值则认为匹配成功
    如果识别到 expected_name，返回 True
    否则继续识别，直到超时
    """
```

在登录时调用该函数：

```
if face_auth("yolanda"):
    登录成功
else:
    人脸认证失败
```

> **Note**
>
> 欧氏距离阈值：

> 人脸识别通常会计算两个人脸特征向量的距离：距离越小，越像同一个人；距离越大，越不像同一个人

```
# 阈值定义
FACE_RECO_THRESHOLD = 0.4
```

3. 代码实现

```
import tkinter as tk
from tkinter import messagebox
import cv2
import dlib
import pandas as pd
import numpy as np
import time
from PIL import Image


# ==========================
# 1. 用户信息配置
# ==========================

USERS = {
    "yolanda": {
        "password": "123456",
        "face_name": "yolanda"
    }
}

FEATURES_CSV_PATH = "data/features_all.csv"
PREDICTOR_PATH = "data/data_dlib/shape_predictor_68_face_landmarks.dat"
FACE_RECO_MODEL_PATH =
"data/data_dlib/dlib_face_recognition_resnet_model_v1.dat"

FACE_RECO_THRESHOLD = 0.45


# ==========================
# 2. 初始化 Dlib 模型
# ==========================

detector = dlib.get_frontal_face_detector()
predictor = dlib.shape_predictor(PREDICTOR_PATH)
face_reco_model = dlib.face_recognition_model_v1(FACE_RECO_MODEL_PATH)

```

```python

# ========================
# 3. 密码认证
# ========================

def check_password(username, password):
    if username not in USERS:
        return False
    return USERS[username]["password"] == password


# ========================
# 4. 读取人脸特征库
# ========================

def load_face_database():
    """
    读取 data/features_all.csv
    返回：
    names: 人名列表
    features: 对应的 128 维特征列表
    """
    data = pd.read_csv(FEATURES_CSV_PATH, header=None, encoding="utf-8")

    names = []
    features = []

    for i in range(data.shape[0]):
        name = str(data.iloc[i, 0])
        feature = data.iloc[i, 1:].astype(float).values

        names.append(name)
        features.append(feature)

    return names, features


# ========================
# 5. 计算欧氏距离
# ========================

def euclidean_distance(feature_1, feature_2):
    return np.linalg.norm(feature_1 - feature_2)


# ========================
# 6. 人脸认证核心函数
```

```python
# ========================

def face_auth(expected_name, timeout_seconds=10):
    """
    打开摄像头进行人脸认证。
    如果在 timeout_seconds 秒内识别到 expected_name，则返回 True。
    否则返回 False。
    """

    names, features = load_face_database()

    cap = cv2.VideoCapture(0)

    if not cap.isOpened():
        messagebox.showerror("错误", "无法打开摄像头")
        return False

    start_time = time.time()
    success_count = 0

    while True:
        ret, frame = cap.read()

        if not ret:
            break

        faces = detector(frame, 1)

        for face in faces:
            shape = predictor(frame, face)
            current_feature = np.array(
                face_reco_model.compute_face_descriptor(frame, shape)
            )

            min_distance = 999
            matched_name = "unknown"

            for i, known_feature in enumerate(features):
                distance = euclidean_distance(current_feature,
known_feature)

                if distance < min_distance:
                    min_distance = distance
                    matched_name = names[i]

            if min_distance < FACE_RECO_THRESHOLD:
                display_name = matched_name
```

```
            else:
                display_name = "unknown"

            # 在画面上画框
            left = face.left()
            top = face.top()
            right = face.right()
            bottom = face.bottom()

            cv2.rectangle(frame, (left, top), (right, bottom), (0, 255, 0), 2)
            cv2.putText(
                frame,
                str(display_name),
                (left, top - 10),
                cv2.FONT_HERSHEY_SIMPLEX,
                0.8,
                (0, 255, 0),
                2
            )

            # 判断是否识别到目标用户
            if display_name == expected_name:
                success_count += 1
            else:
                success_count = 0

            # 连续 3 帧识别成功，才认为认证通过
            if success_count >= 3:
                cap.release()
                cv2.destroyAllWindows()
                return True

        cv2.imshow("Face Authentication - Press Q to quit", frame)

        # 按 q 手动退出
        if cv2.waitKey(1) & 0xFF == ord("q"):
            break

        # 超时退出
        if time.time() - start_time > timeout_seconds:
            break

    cap.release()
    cv2.destroyAllWindows()
    return False

```

```python

# ========================
# 7. 登录按钮事件
# ========================

def login():
    username = entry_username.get().strip()
    password = entry_password.get().strip()

    if username == "" or password == "":
        messagebox.showwarning("提示", "请输入用户名和密码")
        return

    if not check_password(username, password):
        messagebox.showerror("登录失败", "用户名或密码错误")
        return

    messagebox.showinfo("密码认证成功", "密码正确，接下来进行人脸识别")

    expected_face_name = USERS[username]["face_name"]

    if face_auth(expected_face_name):
        messagebox.showinfo("登录成功", "双因子认证成功，登录系统")
        label_status.config(text="状态：登录成功")
    else:
        messagebox.showerror("登录失败", "人脸认证失败")
        label_status.config(text="状态：人脸认证失败")


# ========================
# 8. Tkinter 登录界面
# ========================

window = tk.Tk()
window.title("双因子认证登录系统")
window.geometry("360x240")

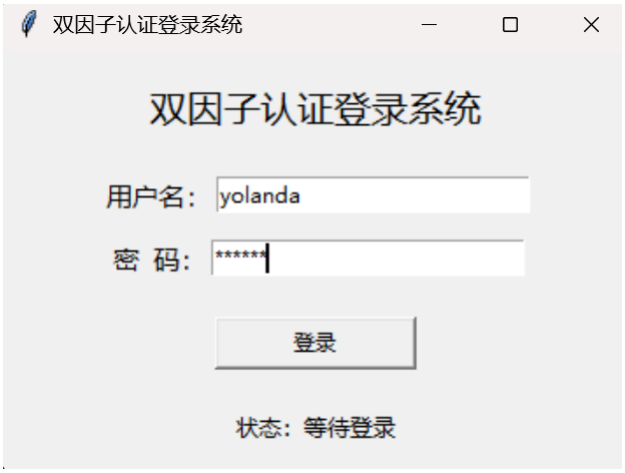
label_title = tk.Label(window, text="双因子认证登录系统", font=("Microsoft YaHei", 16))
label_title.pack(pady=15)

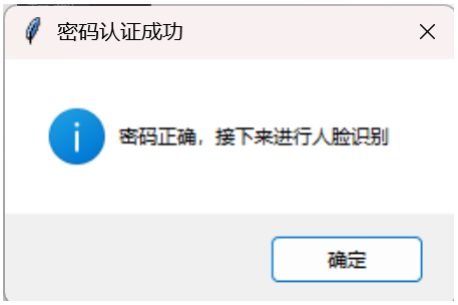
frame_username = tk.Frame(window)
frame_username.pack(pady=5)

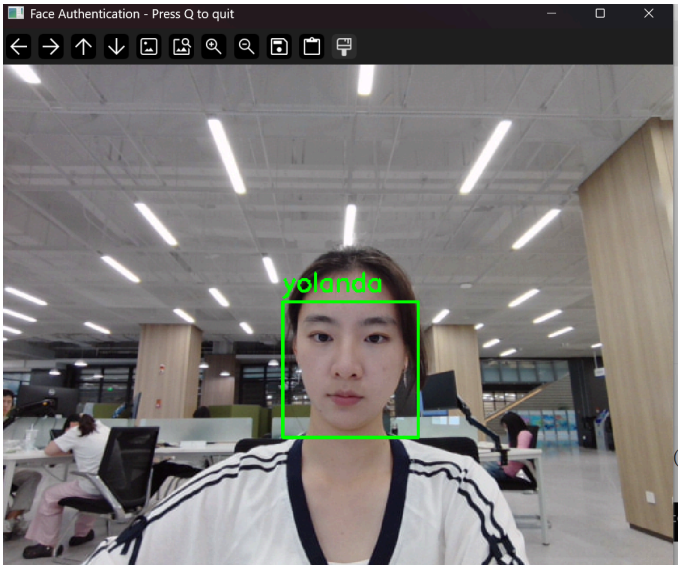
label_username = tk.Label(frame_username, text="用户名：", font=("Microsoft YaHei", 11))
label_username.pack(side=tk.LEFT)
```

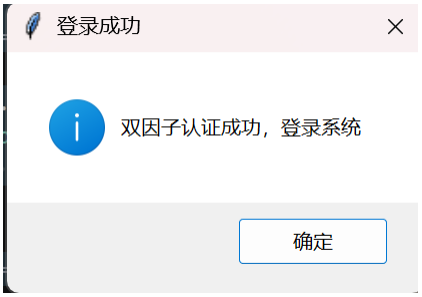
```
221
222 entry_username = tk.Entry(frame_username, width=25)
223 entry_username.pack(side=tk.LEFT)
224
225 frame_password = tk.Frame(window)
226 frame_password.pack(pady=5)
227
228 label_password = tk.Label(frame_password, text="密  码：", font=
    ("Microsoft YaHei", 11))
229 label_password.pack(side=tk.LEFT)
230
231 entry_password = tk.Entry(frame_password, width=25, show="*")
232 entry_password.pack(side=tk.LEFT)
233
234 button_login = tk.Button(window, text="登录", width=15, command=login)
235 button_login.pack(pady=15)
236
237 label_status = tk.Label(window, text="状态：等待登录", font=("Microsoft
    YaHei", 10))
238 label_status.pack(pady=5)
239
240 window.mainloop()
```

登陆界面如下：









五、调试问题

1. 在进行摄像头实时识别时，出现那如下报错：cv2.putText() 需要显示的文字必须是 str 字符串，但程序传进去的是 numpy.float64 类型。

```
(face_lab) D:\yyr\study\aaprofessional\network_security_technology\lab6\Dlib_face_recogn
ition_from_camera>python face_reco_from_camera_ot.py
INFO:root:Faces in Database:  5
Traceback (most recent call last):
  File "face_reco_from_camera_ot.py", line 306, in <module>
    main()
  File "face_reco_from_camera_ot.py", line 302, in main
    Face_Recognizer_con.run()
  File "face_reco_from_camera_ot.py", line 292, in run
    self.process(cap)
  File "face_reco_from_camera_ot.py", line 208, in process
    img_rd = cv2.putText(img_rd, self.current_frame_face_name_list[i],
cv2.error: OpenCV(4.10.0) :-1: error: (-5:Bad argument) in function 'putText'
> Overload resolution failed:
>  - Can't convert object of type 'numpy.float64' to 'str' for 'text'
>  - Can't convert object of type 'numpy.float64' to 'str' for 'text'
```

`INFO:root:Faces in Database: 5` 说明程序已经成功读取到了数据库，说明 `features_all.csv` 是存在的，里面确实有 5 条人脸数据。

问题可能出在：**人脸数据库 CSV 文件里的“姓名列”不是字符串，或者为空，导致 pandas / numpy 把它读成了 float64。**

✨ 原因：检查`features_all.csv` 后发现姓名列并非为空，而是数字。所以可能是项目代码没有兼容新版 pandas / OpenCV，新版 OpenCV 对 `cv2.putText()` 参数类型更严格，必须是字符串，不能自动把数字或 `NaN` 转成字符串。

```
1,-0.08057396858930588,0.093224436044693,0.0083
8107373714447,-0.1879364550113678,0.154721856
6376,-0.03766396641731262,-0.04421749711036682
03301311284303665,0.07693057507276535,0.03240
3333683014,-0.16517576575279236,-0.07671392709
```

👉 将img_rd中的第二个参数改为 如下写法,:

```
    for i in range(self.current_frame_face_cnt):
        # 6.2 Write names under ROI
        name = self.current_frame_face_name_list[i]

        if not isinstance(name, str):
            name = "unknown"
        else:
            name = str(name)

        img_rd = cv2.putText(img_rd, name,
                             tuple([self.current_frame_face_position_list[i][0],
                                    self.current_frame_face_position_list[i][1]]),
                             cv2.FONT_ITALIC, 0.8, (255, 255, 255), 1)
```

2. 人脸识别后都显示unkown

👉 （1）可以通过添加“编号到名字”的映射表

```
# 人脸编号到用户名的映射
# 根据 data/data_faces_from_camera/ 下面的文件夹名称来对应
PERSON_NAME_MAP = {
    "person_1": "yolanda",
    "person_2": "user2",
    "person_3": "user3",
    "person_4": "user4",
    "person_5": "user5",
}

# person_name = person.split('_', 2)[-1]
# features_mean_personX = np.insert(features_mean_personX, 0, person_name, axis=0)

#将上面的内容修改为如下
person_name = PERSON_NAME_MAP.get(person, person)
features_mean_personX = np.insert(features_mean_personX, 0, str(person_name), axis=0)
```

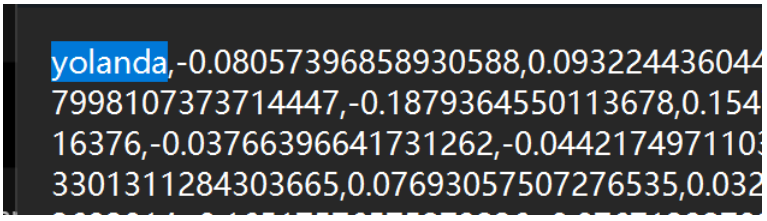
（2）不添加映射表，直接修改文件夹名字

源代码中获取姓名的代码为：`person_name = person.split('_', 2)[-1]`，因此可以将文件

夹名字修改如下：



然后重新运行特征提取脚本。



六、体会与心得

通过本次实验，我成功完成了基于 Dlib 和 OpenCV 的人脸识别系统部署，并实现了结合用户名密码与人脸识别的双因子认证登录功能。实验过程中，我了解了人脸检测、人脸特征提取和特征比对的基本原理，掌握了利用摄像头采集人脸信息、生成特征数据库以及实时识别人脸的方法。

在实验实施过程中，我还遇到了一些环境配置和程序运行问题，例如 OpenCV 显示文字时的数据类型错误等。通过查阅资料和分析代码逻辑，最终成功解决了相关问题。这让我认识到，实际项目开发不仅需要掌握理论知识，还需要具备较强的问题分析和调试能力。

通过实现双因子认证登录系统，我进一步理解了身份认证的安全机制。相比单独使用密码认证，将密码与人脸识别结合能够有效提高系统安全性，降低账号被冒用的风险。总体而言，本次实验加深了我对人脸识别技术和双因子认证原理的理解，提高了我的 Python 编程能力和实践动手能力，也让我认识到安全系统设计需要兼顾功能实现与安全防护。